Report EX2- Hadar Wolf 209800804

5.1

3)

```
Starting KNN Grid Search (5.1)...

Accuracy Table (rows = k, cols = distance metric):
            l1        l2
1     0.967044  0.968043
10    0.961718  0.957723
100   0.923103  0.920107
1000  0.745007  0.741678
3000  0.401798  0.398136
```

5.2

1) **The Trend**

The overall trend is a steep decline in accuracy as the value of k increases.

- The highest accuracies (over 96%) are achieved when k=1 and k=10.
- Accuracy drops significantly when k=1000 (down to 74\%).
- Accuracy is lowest when k=3000 (down to 40\%).

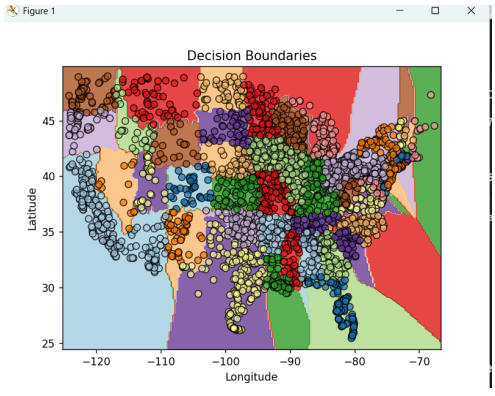
Difference Between Distance Metrics

The overall trend does not change between the different distance metrics (L1 and L2).

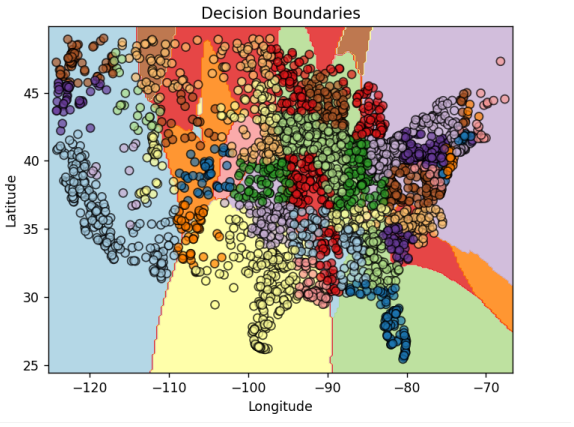
- In every single row, increasing k leads to decreasing accuracy for both L1 and L2.
- **Performance Difference:** The L2 (Euclidean) metric consistently performs slightly better than the L1 (Manhattan) metric across all values of k. This suggests that for these particular features (longitude and latitude), the squared straight-line distance is a better measure of similarity than the distance traveled along the grid axes.

kmax= 1 , kmin = 3000

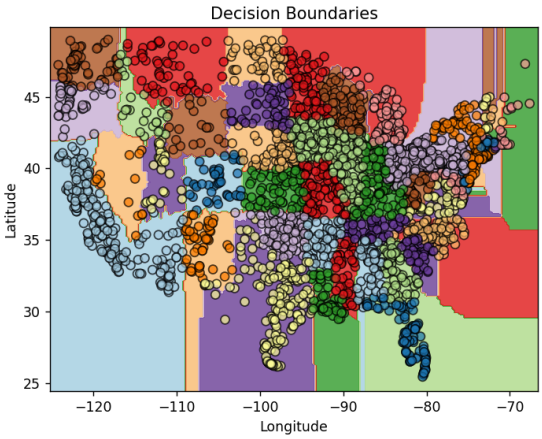
i)



ii)



iii)



a)**Low k=1**

When k=1, the model is very picky and local. It makes a decision for every single tiny area based *only* on the one closest point. This results in boundaries that are highly detailed and rough, uneven edge . The model is essentially drawing a tiny fence around every training data point.

High k=3000

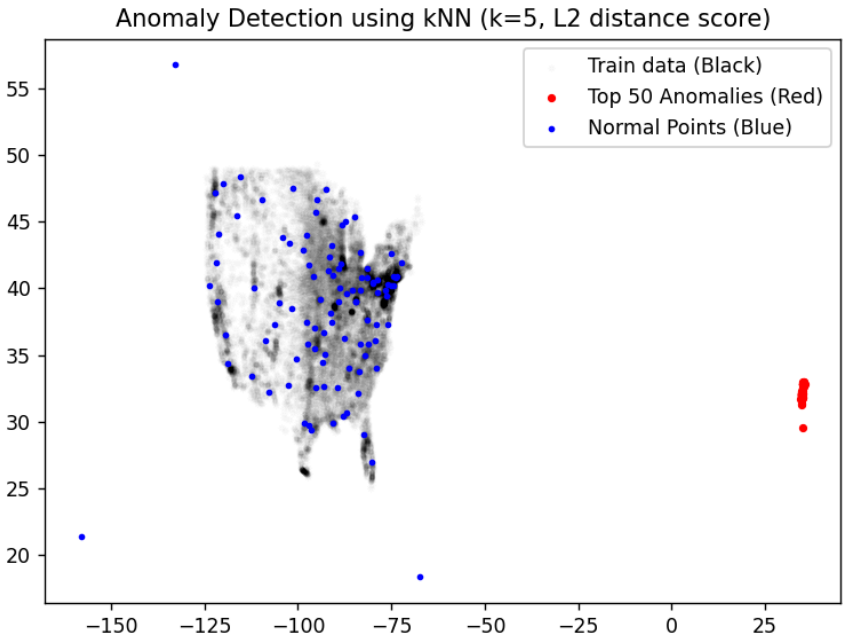
When k=3000, the model is lazy and global. It averages its decision over a massive, wide area. This forces the model to ignore all the small details and create very smooth, simple boundaries . The result is a simple map that misses local features.

The kmax=1 model results in the best accuracy because it has low bias, allowing it to effectively capture the complex, sharp, and localized boundaries (like state lines) that exist within the spatial (longitude/latitude) dataset. Since k=1 only looks at the single closest neighbor, it doesn't average or smooth the decision across large areas. This high fidelity to the local data structure prevents the model from oversimplifying the problem (high bias) and maintains the necessary complexity to accurately distinguish between neighboring regions, which is crucial for high performance in this specific task.

b)The choice between the L2 (Euclidean) and L1 (Manhattan) distance metrics at k=1 fundamentally affects the geometry of the classification space. The boundaries in the L2 plot appear smoother and more natural because Euclidean distance measures the shortest straight line between points, resulting in straight, free-angling boundary segments. In contrast, the boundaries in the L1 plot appear rectilinear, blocky, or stepped because Manhattan distance measures movement restricted to the feature axes (horizontal and vertical), forcing the decision lines to align more closely with the longitude and latitude axes. This difference in measurement

results in distinct boundary shapes even though both models are equally detailed and erratic due to the low k=1 value.

5.3



The plot summarizes the result of using the kNN algorithm to find geographic outliers. We first established the expected pattern by treating all the training data (the faint black dots) as the baseline for "normal" locations. To find the anomalies, we calculated an Anomaly Score for every test point by summing the L2 distance to its 5 nearest neighbors in that black training set. This score directly measures how isolated a point is. We then identified the 50 test points with the highest scores, coloring them red (Anomalies), and colored the rest blue (Normal). The final plot confirms the method works perfectly because the red points are visually concentrated in the blank, empty areas of the map, signifying locations that are unusually distant from any established state data.

5.4

The anomalies (red dots) are structurally different because they have the highest Anomaly Scores, meaning they are the most geographically isolated points in the entire test set. We calculated this score by summing the $\mathbf{L2}$ distance to their $\mathbf{5}$ nearest neighbors in the training data. The red dots are concentrated in the empty areas of the map, proving they sit far outside the established pattern of normal locations. Conversely, the normal points (blue dots) have low scores because they are safely clustered within the dense training data boundaries.

6.1

Decision Tree Grid Search Results (Task 6.1 - Sample):

max_depth	max_leaf_nodes	acc_train	acc_val	acc_test
100	1000	1	0.973996	0.972703
20	1000	1	0.973996	0.972703
50	1000	1	0.973996	0.972703
20	100	0.952299	0.935303	0.935087
100	100	0.952299	0.935303	0.935087
50	100	0.952299	0.935303	0.935087
10	1000	0.943716	0.93031	0.927097
10	100	0.927278	0.915332	0.915113
50	50	0.856638	0.840649	0.830559
20	50	0.856638	0.840649	0.830559
100	50	0.856638	0.840649	0.830559
10	50	0.844257	0.829623	0.8249
6	1000	0.596494	0.586228	0.585553
6	50	0.596494	0.586228	0.585553
6	100	0.596494	0.586228	0.585553
4	50	0.366781	0.363844	0.361851
4	100	0.366781	0.363844	0.361851
4	1000	0.366781	0.363844	0.361851
2	1000	0.190231	0.189931	0.189747
2	50	0.190231	0.189931	0.189747
2	100	0.190231	0.189931	0.189747
1	100	0.11366	0.114	0.113848
1	1000	0.11366	0.114	0.113848
1	50	0.11366	0.114	0.113848

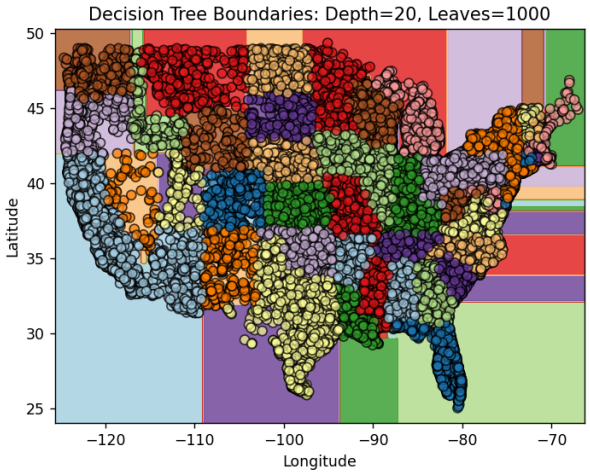
6.2

1. The highest validation accuracy is 0.973, achieved by the trees with 1000 leaf nodes and depths 20, 50, and 100. The corresponding test accuracy for these best-validation models is 0.972.

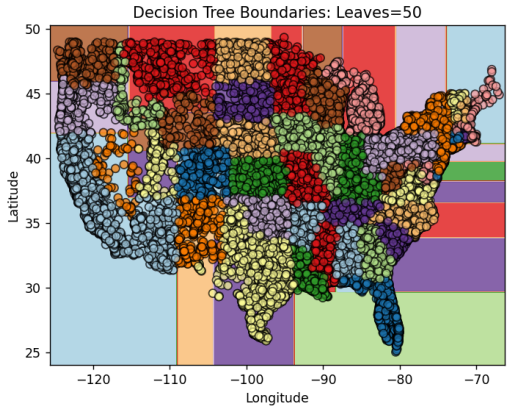
2.The Decision Tree is highly successful at generalization, achieving about 97\% accuracy on unseen test data. However, the perfect 100\% training accuracy shows a slight degree of overfitting because the model memorized noise specific to the training set. We can confidently choose the best tree using our validation set because the validation accuracy is nearly identical to the final test accuracy across all models, proving that the validation set provides a reliable estimate of the tree's true performance.

3.While a leaf node predicts a single state, the data is typically not perfectly clean. To achieve perfect accuracy, a Decision Tree requires a separate, pure leaf node for every unique region of space belonging to a specific class. Because the map data contains complex, irregular borders and likely includes noisy points near those boundaries, the tree requires significantly more than just 50 leaves to accurately define the messy geographical boundaries and isolate every data point correctly. The Decision Tree must make hundreds or even thousands of small, granular cuts (which necessitates generating many more than 50 leaves) to fully capture the true complexity of the data and achieve near-perfect separation.

4

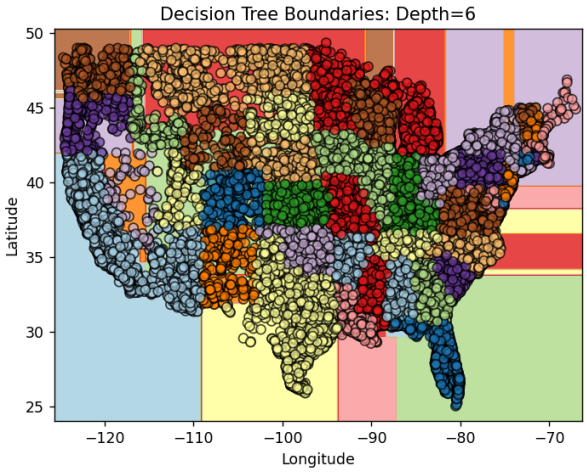


The visualization confirms that Decision Trees partition the space into axis-aligned rectangles. Because Decision Trees split strictly on individual features (either longitude or latitude), the map is divided only by vertical and horizontal lines. This sequential splitting results in sharp, block-like regions, making each class appear as a collection of rectangular tiles perfectly aligned with the coordinate axes.



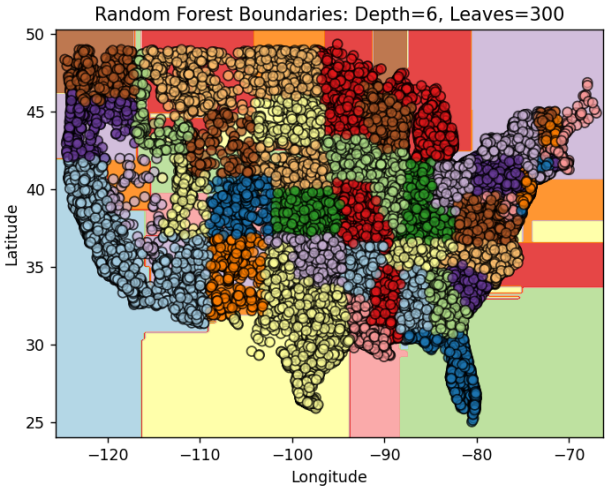
5.

The model restricted to 50 leaf nodes performs significantly worse (Accuracy approx 84\%) than the overall best model (Accuracy approx 97\%). This happens because the model is too simple; the restriction forces the tree to oversimplify the state boundaries (high bias), creating large, blocky regions instead of the detailed, accurate cuts needed to separate the complex geographic data. Therefore, the constraint prevents the tree from achieving the necessary complexity to generalize successfully.

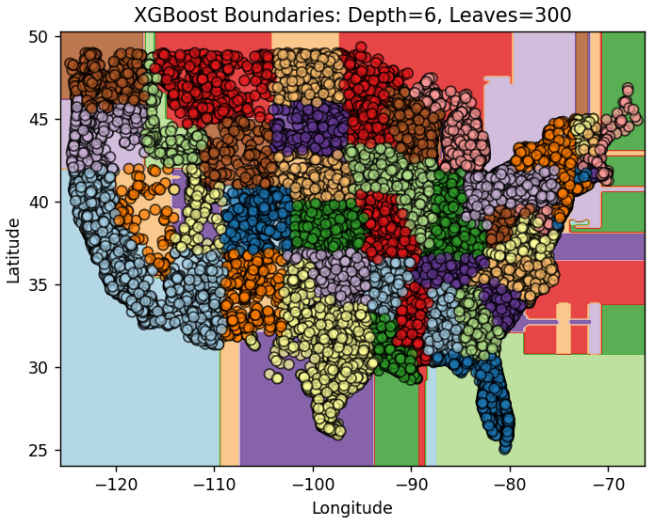


6.
Restricting the tree's maximal depth to 6 forces a significant oversimplification of the map. Compared to the complex boundaries of the overall best model, this shallow tree creates boundaries that are much larger, simpler, and blockier. The limited depth prevents the tree from making enough sequential cuts to accurately trace the fine geographic borders, resulting in high bias and a substantial drop in accuracy, as the model is too simple to represent the complex data.

7.The Random Forest model is more expressive than the single best Decision Tree, even though its individual trees are shallower (Depth 6 vs. Depth 20+). The Random Forest achieves this superior expressiveness by averaging the predictions of 300 different trees, a technique that greatly reduces the variance inherent in any single, deep tree. The visualization directly supports this by showing that the Random Forest's decision boundaries are smoother and more continuous, proving it generalizes the complex state borders better by effectively averaging out the rough, blocky edges of its simple component trees.



8)



The XGBoost model is more successful than the Random Forest (RF) because it achieves a slightly higher prediction accuracy. The two models differ fundamentally in how they learn: while RF achieves smoothness by simply averaging its 300 independent trees (a process called Bagging), XGBoost uses Boosting, fitting each new tree sequentially to specifically correct the mistakes of the previous ones. This focused, error-correcting process allows XGBoost to produce classification boundaries that are visually cleaner and more precise

AI Tools

I used the AI tool in a few ways to help with this project. I used it to understand some of the harder questions in the assignment and give a description for my code.